SAGAPI v2.0 — Manual de ejecución

Instalación, comandos, configuración, interpretación de resultados y reemplazo de datos simulados por reales

Contenido

1. Requisitos
2. Estructura
3. Comandos
4. Pruebas
5. Configuración (en la base, sin tocar el código)
6. Experimentos
7. Reemplazar datos simulados por datos reales
8. Interpretación de resultados
9. Problemas frecuentes

1. Requisitos

- Python 3.12 o 3.13 (probado en 3.13 y, en un entorno limpio creado desde el ZIP, en 3.12). Python 3.11 no sirve: numpy 2.5.3 no tiene paquetes para esa versión.
- Paquetes con versiones exactas en `01_CODIGO_PYTHON/requirements.txt`: numpy, scipy (incluye HiGHS), pandas, openpyxl, matplotlib y pytest. No se usa ningún solucionador comercial ni ninguna API privada.
- Solo para regenerar los PDF: pandoc y Chromium (`herramientas/generar_pdfs.py`). Solo para recalcular las fórmulas del Excel: LibreOffice.

```
cd SAGAPI_MILP_VERSION_OPTIMIZADA_FINAL
python -m venv .venv && source .venv/bin/activate      # Windows: .venv\Scripts\activate
pip install -r 01_CODIGO_PYTHON/requirements.txt
```

2. Estructura

Carpeta	Contenido
`01_CODIGO_PYTHON/`	`Programa_principal.py` (punto de entrada), paquete `sagapi/`, `herramientas/`, `requirements.txt`
`02_BASE_DATOS/`	`SAGAPI_BASE_CORREGIDA_FINAL.xlsx` (única base que lee el programa), `Diccionario_de_datos.xlsx`, `version_anterior/`
`03_MODELO_MATEMATICO/`	`Formulacion_MILP_Actualizada.pdf`
`04_RESULTADOS/`	tablas de las corridas: `Resultados_MILP`, `Comparacion_MILP_FIFO`, `Indicadores_KPI`, `Escenarios`; libro `Resultados_SAGAPI_v2.xlsx`; registro de ejecución
`05_GRAFICOS/`	figuras generadas solo a partir de `04_RESULTADOS`
`06_AUDITORIA_Y_VALIDACION/`	informe de correcciones, matriz de errores, reporte de pruebas y evidencias (incluida la versión inicial v1.4.0)
`07_DOCUMENTACION/`	este manual, el informe técnico, el registro de cambios, las limitaciones y las fuentes Markdown de los PDF
`tests/`	pruebas automáticas (pytest)

Paquete `sagapi/` por capas: `datos/` (esquema, lectura estricta, controles, modelo de datos) → `instancia.py` (conjuntos y parámetros) → `modelo/milp.py` (formulación) → `solver.py` (HiGHS y estados) → `optimizacion.py` (lexicográfico) → `estado.py` y `simulacion.py` (horizonte rodante) → `reglas.py` (FIFO, EDD, reacción convencional) → `personal.py` (certificado, asignación, reparación) → `verificacion.py` (verificador independiente) → `indicadores.py` (KPI) → `reportes/` (gráficos y libro) → `cli.py` (línea de comandos) y `experimentos.py`.

3. Comandos

Todos se ejecutan desde `01_CODIGO_PYTHON/`. Las rutas por defecto son relativas a la carpeta del proyecto, de modo que el ZIP se puede descomprimir en cualquier lugar.

Comando	Qué hace	Salida
`python Programa_principal.py validar`	Controla la calidad de la base (Q00–Q17).	Informe en pantalla; código 2 si hay errores
`python Programa_principal.py programar --plan MILP --reaccion MILP --escenario S20A`	Simula el año por eventos con la combinación indicada, asigna trabajadores y verifica.	`04_RESULTADOS/*/…_S20A_MILP_MILP.csv`
`python Programa_principal.py caso --escenario S06A`	Compara CONV, EDD y MILP sobre el mismo subproblema de la primera avería.	`caso_S06A_*.csv`, `E2b_reaccion_evento.csv`
`python Programa_principal.py experimentos [--solo E0,E1]`	Ejecuta E0–E5 (sección 6).	todas las tablas de `04_RESULTADOS`
`python Programa_principal.py reportes`	Gráficos y libro de resultados a partir de lo ya calculado.	`05_GRAFICOS/`, `Resultados_SAGAPI_v2.xlsx`
`python Programa_principal.py todo`	`experimentos` + `reportes`.	todo lo anterior

Opciones comunes: `--base RUTA` (otra base), `--salida CARPETA`, `--tl SEG` (límite por fase de HiGHS; por defecto `limite_computo_s`), `--regla-objetivo {TARDANZA_PONDERADA, PENALIDAD_FIJA, PENALIDAD_DIARIA_CON_TOPE}`, `--log DEBUG`, `--log-archivo RUTA`. En `programar`: `--plan {FIFO, EDD, MILP}`, `--reaccion {CONV, EDD, MILP}`, `--escenario` (cualquier `escenario_id` de `24_ESCENARIOS_FALLA_SIM`, o `S00` sin averías), `--congelar H` (horizonte congelado en h) y `--sin-personal`.

Códigos de salida: 0 correcto; 1 error no previsto (traza completa en el registro); 2 base de datos con errores (no se optimiza); 3 programa infactible o violación detectada por el verificador.

4. Pruebas

```
cd SAGAPI_MILP_VERSION_OPTIMIZADA_FINAL
python -m pytest tests -q
```

La batería completa (enumeración exhaustiva, una prueba por restricción, calidad de datos, personal, horizonte rodante, regresión con v1.4, estados del solucionador y línea de comandos) dura unos pocos minutos. El resultado de la última ejecución está en `06_AUDITORIA_Y_VALIDACION/Reporte_Pruebas/`.

5. Configuración (en la base, sin tocar el código)

Hoja · parámetro	Efecto	Valor actual
`28_CONFIG_MODELO · regla_objetivo`	Jerarquía de objetivos (ver formulación, sección 8)	`TARDANZA_PONDERADA`
`28_CONFIG_MODELO · beta_reasignacion`	Peso de una reasignación en la estabilidad (h)	8
`28_CONFIG_MODELO · limite_computo_s`, `brecha_relativa`	Límite por fase y brecha de HiGHS	60 s, 1e-5
`28_CONFIG_MODELO · limite_asignacion_personal_s`	Límite del MILP de asignación de personal	120 s
`11_PARAMETROS · lote_no_cruza`	Continuidad de operaciones no interrumpibles: `TRAMO` (4 h) o `JORNADA` (8 h)	`TRAMO`

Hoja · parámetro	Efecto	Valor actual
`11_PARAMETROS · horizonte_congelado_h`	Decisiones congeladas al reprogramar	0
`11_PARAMETROS · aplicar_feriados`	Descontar feriados nacionales del calendario	0
`11_PARAMETROS · jornada_productiva`, `hora_liberacion`, `hora_vencimiento`	Eje de tiempo y horas de los hitos	08:00–12:00 y 13:00–17:00; 08:00; 17:00
`11_PARAMETROS · fuente_habilidades`, `fuente_pedidos`	`SIMULADA` o `REAL` (sección 7)	`SIMULADA`

Después de modificar la base, ejecute siempre `validar`. Los errores se informan con hoja, fila y columna, y el programa no optimiza mientras existan.

6. Experimentos

Id	Contenido	Archivos
E0	Calidad de datos, tamaño de la instancia, certificado laboral, carga por recurso, tardanza inevitable, cuello de botella	`Resultados_MILP/E0_*`, `Indicadores_KPI/E0_*`
E1	Año completo S00 y S20A con FIFO+CONV, FIFO+MILP, EDD+EDD, MILP+CONV y MILP+MILP	`Comparacion_MILP_FIFO/E1_*`, `programa_*`, `eventos_*`
E2	Escenarios de avería de `24_ESCENARIOS_FALLA_SIM` sobre el plan FIFO (sistema completo, Δ frente a S00)	`Comparacion_MILP_FIFO/E2_escenarios_falla.csv`
E2b	Reacción sobre el mismo subproblema (S06A, S13A, S19A)	`E2b_reaccion_evento.csv`, `caso_*`
E3	Escenarios operativos de `27_ESCENARIOS_OPERATIVOS_SIM`	`Escenarios/E3_escenarios_operativos.csv`
E4	Sensibilidad de `26_SENSIBILIDAD_SIM`	`Escenarios/E4_sensibilidad.csv`
E5	Rendimiento v2.0 frente a v1.4 en el mismo subproblema; presolve	`Resultados_MILP/E5_rendimiento.csv`

Todo escenario es SIMULADO y así se rotula en las tablas (columna `naturaleza`).

7. Reemplazar datos simulados por datos reales

Matriz de habilidades. Registre en `16_REG_HABILIDADES` una fila por trabajador y recurso: `operario_id`, `recurso_id`, `nivel_0_3`, `opera_solo_1_0`, `validado_por` y `fecha_validacion`. Un trabajador queda habilitado para un recurso si `nivel_0_3 ≥ 2` y `opera_solo_1_0 = 1`. Luego cambie `fuente_habilidades` a `REAL` y ejecute `validar`. Si el certificado de cobertura deja de cumplirse, el programa pasa automáticamente al modo de reparación laboral y lo informa (`modo_personal = REPARACION_LABORAL`). Si algún recurso queda sin personal habilitado, se detiene con un mensaje

explícito. La prueba `test_matriz_de_habilidades_real_reemplaza_a_la_simulada` comprueba este mecanismo.

Personas por operación. Columna `trabajadores_requeridos` de `06_RUTAS` (hoy = 1, supuesto heredado de v1.4).

Pedidos. Registre los pedidos en `12_REG_ENTREGAS` y cambie `fuente_pedidos` a `REAL`.

Tiempos. Cronometre en `15_REG_TIEMPOS` y actualice `min_fijos_lote`, `min_por_unidad` y `origen_tiempo` en `06_RUTAS`.

Paradas. Registre las paradas reales en `13_REG_PARADAS`; los escenarios de averías siguen siendo simulados mientras no se carguen eventos reales.

8. Interpretación de resultados

- **Estado de solución.** `OPTIMO`: HiGHS demostró la optimalidad de cada fase y el vector pasó la verificación algebraica. `FACTIBLE_LIMITE_TIEMPO`: solución factible sin prueba de optimalidad. `HEURISTICA_VERIFICADA`: solución de regla (o con reparación laboral) que es factible pero no es óptima. Ningún resultado se declara óptimo sin respaldo del solucionador.
- **Verificación independiente.** `verificacion_violaciones = 0` significa que el programa cumple asignación, capacidad, precedencia, liberación, ventanas, continuidad y personal según un verificador que no reutiliza el MILP. **Esto no equivale a una validación en planta.**
- **KPI.** Fórmulas, unidades y procedencia en `04_RESULTADOS/Indicadores_KPI/KPI_definiciones.csv`. Los montos en S/ son estimaciones.

9. Problemas frecuentes

Síntoma	Causa y solución
`ErrorDatos … no se optimiza` (código 2)	La base tiene errores: corrija las celdas indicadas y vuelva a ejecutar `validar`.
`personal insuficiente` (código 3)	Algún recurso no tiene trabajadores habilitados disponibles: revise la matriz o las ausencias del escenario.
`operación no interrumpible … excede el tramo`	Una operación con `interrumpible_en_pausas = 0` dura más que el tramo continuo: revise el tiempo o use `lote_no_cruza = JORNADA`.
Estado `FACTIBLE_LIMITE_TIEMPO`	Aumente `--tl` o `limite_computo_s`; la solución es válida, pero no hay prueba de optimalidad.
Las fórmulas del Excel aparecen sin valores tras editarlo con otra herramienta	`python 01_CODIGO_PYTHON/herramientas/recalcular_excel.py 02_BASE_DATOS/SAGAPI_BASE_CORREGIDA_FINAL.xlsx` (requiere LibreOffice).